

Correction — TD/TP 5

State, TimedState et Trajectory

MAM4 — Programmation C++

Cette correction donne le code minimal et les réponses directes. Les traces temporaires de construction ne font pas partie de la version finale.

1. State et la règle de zéro

state.hpp :

```
#ifndef SEANCE5_STATE_HPP
#define SEANCE5_STATE_HPP
#include <cstdlib>
#include <vector>

class State {
public:
    explicit State(std::size_t dimension,
                  double initial_value = 0.0);
    double& operator[](std::size_t index);
    const double& operator[](std::size_t index) const;
    double& at(std::size_t index);
    const double& at(std::size_t index) const;
    std::size_t size() const noexcept;
private:
    std::vector<double> values_;
};

double l2_norm(const State& state);
void scale(State& state, double factor) noexcept;
#endif
```

state.cpp — opérations essentielles :

```
State::State(std::size_t dimension, double initial_value)
    : values_(dimension, initial_value) {}

double& State::operator[](std::size_t i) { return values_[i]; }
const double& State::operator[](std::size_t i) const
{ return values_[i]; }
double& State::at(std::size_t i) { return values_.at(i); }
const double& State::at(std::size_t i) const
{ return values_.at(i); }
std::size_t State::size() const noexcept { return values_.size(); }

double l2_norm(const State& state)
{
    double sum{};
    for (std::size_t i{}; i < state.size(); ++i)
        sum += state[i] * state[i];
    return std::sqrt(sum);
}

void scale(State& state, double factor) noexcept
{
    for (std::size_t i{}; i < state.size(); ++i)
        state[i] *= factor;
}
```

Réponse. State suit la règle de zéro : vector sait déjà se détruire, se copier et se déplacer. Une copie de State possède son propre vector.

2. TimedState

timed_state.hpp :

```
class TimedState : public State {
public:
    TimedState(std::size_t dimension, double time);
    double time() const noexcept;
    double first() const;
private:
    double time_{};
};
```

timed_state.cpp :

```
TimedState::TimedState(std::size_t dimension, double time)
    : State{dimension}, time_{time} {}

double TimedState::time() const noexcept { return time_; }
double TimedState::first() const { return (*this)[0]; }
```

Réponse. Le sous-objet `State` est construit avant `time_`. La destruction suit l'ordre inverse.

Réponse. `values_` reste privé de `State`. Le dérivé passe par `operator[]` ou `size()`, ce qui laisse `State` contrôler sa représentation.

3. Référence de base et algorithmes existants

```
TimedState u{3, 1.0};
u[0] = 3.0; u[1] = 4.0;

const State& view{u};
assert(l2_norm(view) == 5.0);
scale(u, 2.0);
assert(l2_norm(u) == 10.0);
assert(u.time() == 1.0);
```

Réponse. `const State&` ne crée aucune copie. La référence observe le sous-objet `State` du même `TimedState`. `l2_norm` ignore simplement l'instant.

À retenir. Héritage public : un `TimedState` doit rester utilisable comme un `State`.

4. Slicing

```
State snapshot{u};
```

Réponse. `snapshot` est un nouvel objet dont le type est exactement `State`. Sa partie numérique est copiée ; il ne possède pas de membre `time_`. C'est le slicing.

<code>const State& view{u}</code>	même objet, aucune copie
<code>State snapshot{u}</code>	nouvel objet de base

Le slicing est légal ; il devient un problème lorsque l'intention était de conserver l'objet complet.

5. Trajectory par composition

trajectory.hpp :

```
class Trajectory {
public:
    void append(TimedState state);
    const TimedState& at(std::size_t index) const;
    std::size_t size() const noexcept;
private:
    std::vector<TimedState> states_;
};
```

trajectory.cpp :

```
void Trajectory::append(TimedState state)
{
    if (!states_.empty()
        && state.time() <= states_.back().time())
        throw std::invalid_argument{
            "times must be strictly increasing"};

    states_.push_back(std::move(state));
}

const TimedState& Trajectory::at(std::size_t i) const
{ return states_.at(i); }

std::size_t Trajectory::size() const noexcept
{ return states_.size(); }
```

Réponse. Trajectory possède un vector : elle n'est pas un vector. Son interface **append** peut imposer l'invariant « temps strictement croissants » et sa représentation pourra changer.

Bilan direct

possède-un	composition : un objet contient un autre objet
est-un	héritage public : le dérivé est substituable à la base
référence de base	même objet complet, sans copie
valeur de base	nouvel objet de base, partie dérivée absente
private	la base garde le contrôle de ses données
règle de zéro	membres et base composent leurs bonnes opérations
